Cursul 2

Grafică pe calculator

2025

Cuprins

1. Matrici de Transformare
   - Scalarea
   - Translația
   - Rotația
     - Gimbal Lock
2. Local $\longrightarrow$ Global
3. Matricea modelului
   - Compunerea transformărilor
4. Coordonatele de vizualizare

Matrici de Transformare

Transformări geometrice

Pot fi scrise simplu folosind calculul matricial.

rotația, scalarea, translația și proiecția pot fi scrie ca multiplicare între matrici.

Transformări liniare

Exemplu în plan

considerăm un vector $P = \begin{pmatrix} x \\ y \end{pmatrix}$

Înmulțim coordonatele acestui punct cu matricea $A \in \mathcal{M}_{2,2}$

$$A \cdot P = \begin{pmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} a_{00} \cdot x + a_{01} \cdot y \\ a_{10} \cdot x + a_{11} \cdot y \end{pmatrix}$$

Legătura între matrici și coordonate

Fiecare linie a matricii corespunde unei axe de coordonate:

$$\begin{pmatrix} c_{00} & c_{01} & c_{02} \\ c_{10} & c_{11} & c_{12} \\ c_{20} & c_{21} & c_{22} \end{pmatrix} \begin{matrix} \longrightarrow axa\ x \\ \longrightarrow axa\ y \\ \longrightarrow axa\ z \end{matrix}$$

Matrici 4×4

În practică vom folosi matrici 4×4!

Motive:

- Pentru a putea scrie matricea perspectivei!
- Pentru a putea scrie translația ca o înmulțire de matrici!

Pentru a putea opera cu aceste matrici se extind coordonatele în 3D cu încă o valoare $w = 1$ (punctul omogen):

$$P = (x, y, z, w = 1)$$

Matricea $\begin{pmatrix} M_{00} & M_{01} & M_{02} & 0 \\ M_{10} & M_{11} & M_{12} & 0 \\ M_{20} & M_{21} & M_{22} & 0 \\ M_{30} & M_{31} & M_{32} & 1 \end{pmatrix}$

va lăsa a patra coordonată a punctului omogen neschimbată.

Scalarea

$$\begin{bmatrix} S_1 & 0 & 0 & 0 \\ 0 & S_2 & 0 & 0 \\ 0 & 0 & S_3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} S_1 \cdot x \\ S_2 \cdot y \\ S_3 \cdot z \\ 1 \end{bmatrix}$$

Unde avem

- o matrice de scalare cu factorii de scalare S_1, S_2, S_3 pe diagonală.
- un vector ce reprezintă un punct 3D în coordonate omogene.
- rezultatul operației de scalare.

Translația

Considerăm un vector de translație (T_x, T_y, T_z). Translația poate fi scrisă matricial:

$$\begin{bmatrix} 1 & 0 & 0 & T_x \\ 0 & 1 & 0 & T_y \\ 0 & 0 & 1 & T_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x + T_x \\ y + T_y \\ z + T_z \\ 1 \end{pmatrix}$$

Observăm că se folosesc **coordonatele omogene** ale punctului

- translația nu poate fi scrisă matricial altfel.

Matricea de rotație

Considerăm un punct $P = (1, 0, 0)$ pe care vrem să îl rotim în punctul $P_T = (0, 1, 0)$.

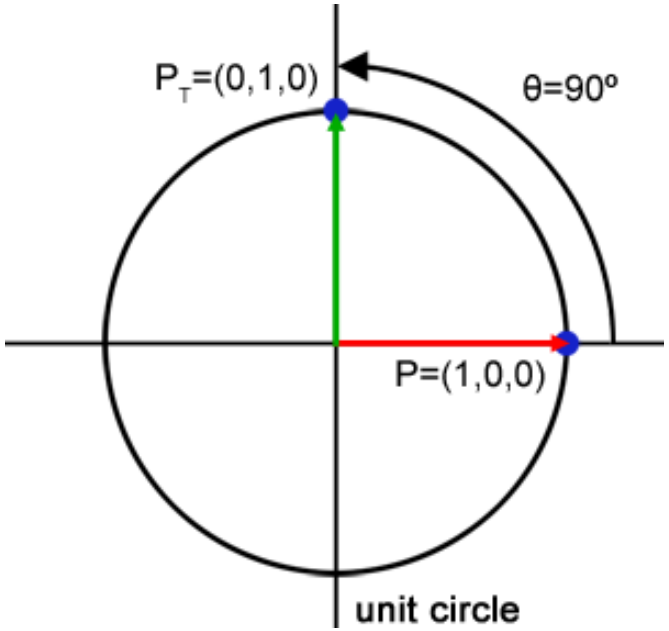


Figure: O rotație de 90 de grade în sens invers acelor de ceasornic.

Matricea de rotație

Observăm că putem scrie operația ca o înmulțire matricială:

$$P_T.x = P.x * R_{00} + P.y * R_{10} + P.z * R_{20}$$

$$P_T.y = P.x * R_{01} + P.y * R_{11} + P.z * R_{21}$$

$$P_T.z = P.x * R_{02} + P.y * R_{12} + P.z * R_{22}$$

deci $P_T = R \cdot P$.

În acest caz:

$$R = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Matricea de rotație

În cazul mai general (o rotație de unghi θ în jurul axei Z):

$$R_Z(\theta) = \begin{pmatrix} cos\theta & sin\theta & 0 \\ sin\theta & cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

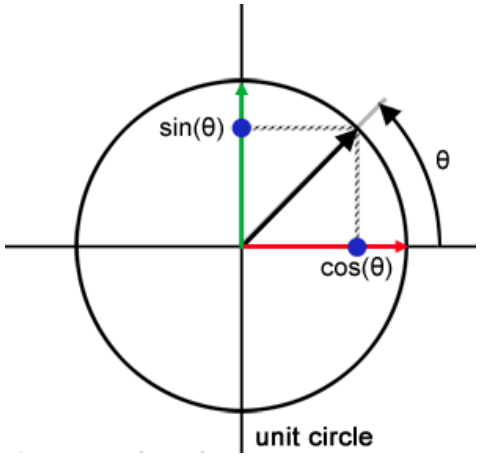


Figure: Cosinusul și sinusul pot fi folosite pentru a determina coordonatele unui punct pe axele x și y ale cercului unitar.

Matricea de rotație

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{pmatrix}$$

$$R_y(\theta) = \begin{pmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{pmatrix}$$

O rotație după două axe se face înmulțind matricile de rotație corespunzătoare

$$R_{XY} = R_X * R_Y$$

Atenție la ordinea rotațiilor!! CONTEAZĂ!!!

Matricile de Rotație

puncte omogene

- rotație după O_x

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ \cos\theta \cdot y - \sin\theta \cdot z \\ \sin\theta \cdot y + \cos\theta \cdot z \\ 1 \end{pmatrix}$$

- rotație după O_y

$$\begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} \cos\theta \cdot x + \sin\theta \cdot z \\ y \\ -\sin\theta \cdot x + \cos\theta \cdot z \\ 1 \end{pmatrix}$$

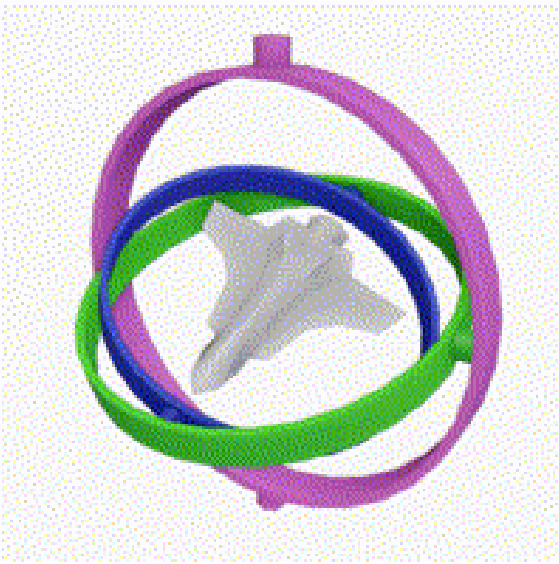
- rotație după O_z

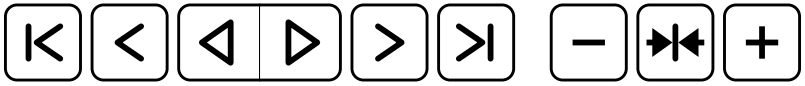
$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} \cos\theta \cdot x - \sin\theta \cdot y \\ \sin\theta \cdot x + \cos\theta \cdot y \\ z \\ 1 \end{pmatrix}$$

O PROBLEMĂ CU ROTAȚIILE!!

Gimbal lock

În anumite cazuri se poate pierde un grad de libertate rotind un obiect.





Aici axa O_x a navetei ajunge aliniată cu axa O_y globală. Naveta atunci nu mai poate vira la stânga sau la dreapta.
Caz real: un incident în timpul misiunii lui Apollo 11 pe Lună.

Soluții

A. Rotație după o axă arbitrară

Considerăm o axă arbitrară (R_x, R_y, R_z) și executăm următoarea rotație:

$$\begin{bmatrix} \cos\theta + {R_x}^2(1-\cos\theta) & R_xR_y(1-\cos\theta) - R_z\sin\theta & R_xR_z(1-\cos\theta) + R_y\sin\theta & 0 \\ R_yR_x(1-\cos\theta) + R_z\sin\theta & \cos\theta + {R_y}^2(1-\cos\theta) & R_yR_z(1-\cos\theta) - R_x\sin\theta & 0 \\ R_zR_x(1-\cos\theta) - R_y\sin\theta & R_zR_y(1-\cos\theta) + R_x\sin\theta & \cos\theta + {R_z}^2(1-\cos\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

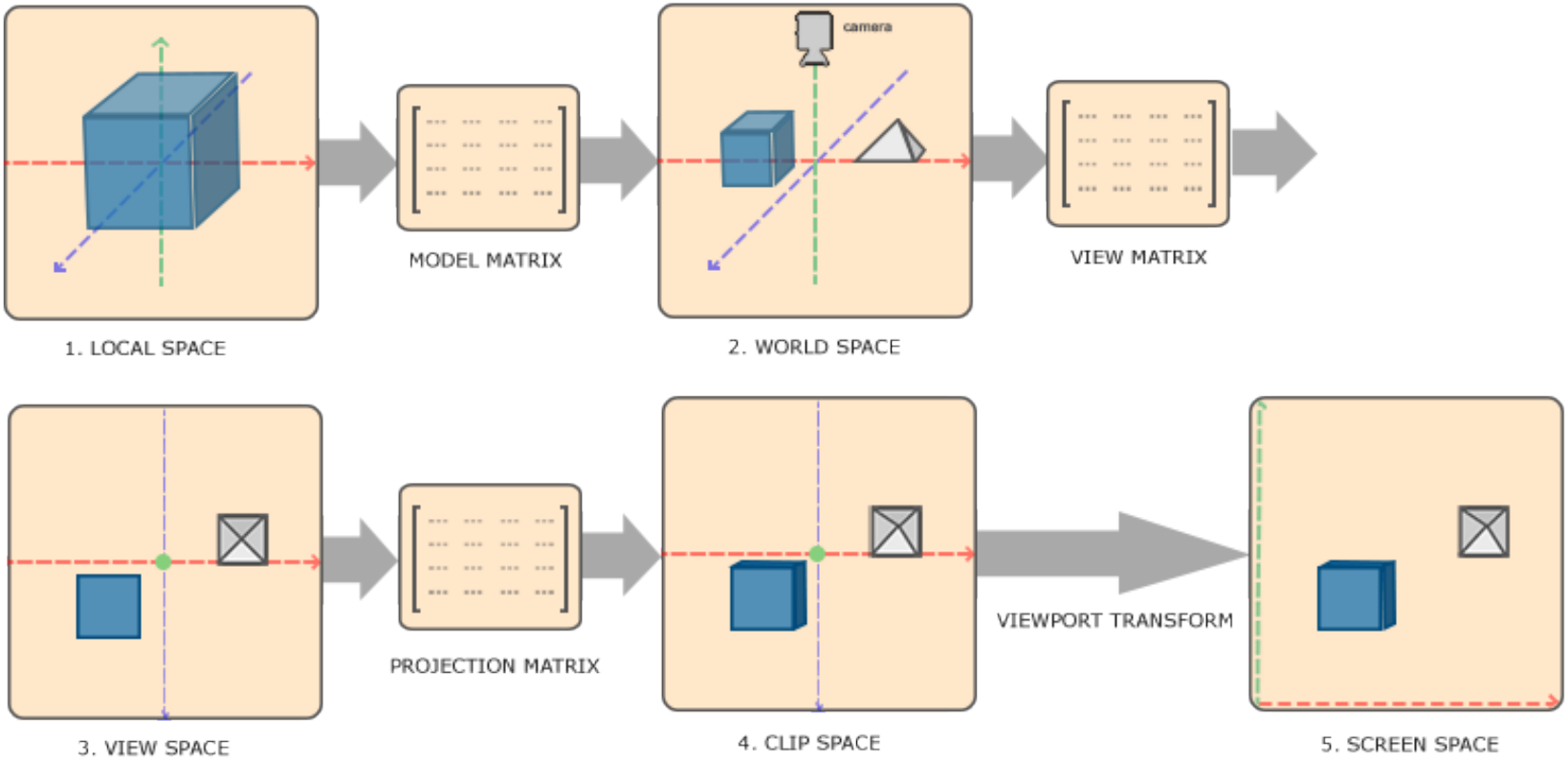
B. Folosim pentru rotații cuaternioni!

oferă o notație matematică convenabilă pentru reprezentarea orientărilor și rotațiilor spațiale ale elementelor în spațiul tridimensional.

C. Mai adăugăm un gimbal

Matrici de transformare

de la local spre global

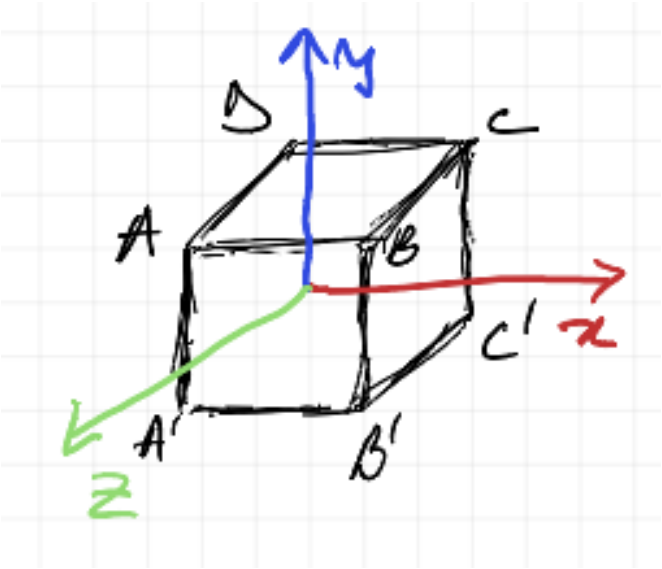


1. **Coordonatele locale**: sunt coordonatele obiectului în raport cu originea sa locală.
2. **Transformarea în coordonate globale**: coordonatele locale sunt convertite în coordonate globale.
3. **Transformarea în coordonate de vizualizare**: coordonatele globale sunt transformate astfel încât fiecare punct să fie perceput din perspectiva camerei.
4. **Proiecția în coordonate de decupare (clip coordinates)**: coordonatele din spațiul de vizualizare sunt proiectate într-un interval standardizat între -1.0 și 1.0. Această etapă determină ce vârfuri (vertex-uri) vor fi vizibile pe ecran.
5. **Transformarea în coordonate de ecran**: coordonatele de decupare sunt transformate în coordonatele finale - ajusteăm coordonatele din intervalul -1.0 la 1.0 în funcție de dimensiunile definite de `glViewport`. Rezultate sunt trimise rasterizatorului pentru a fi convertite în fragmente (pixeli colorați).

Matricea modelului

O matrice de transformare care translatează, scalează și/sau rotește obiectul tău pentru a-l plasa în scena globală într-o locație/orientare căreia îi aparțin.

Exemplu: avem un cub, cu centrul lui în Originea sistemului de coordonate global $O(0, 0, 0)$



Vrem să îl mutăm la poziția (0.5, 0.5, -4) și să îl rotim 45° în jurul axei O_y.

$A = (-0.5, 0.5, 0.5)$
$B = (0.5, 0.5, 0.5)$
$C = (0.5, 0.5, -0.5)$
...

Ordinea transfromărilor

ATENȚIE!!!!!! Ordinea transformărilor contează!

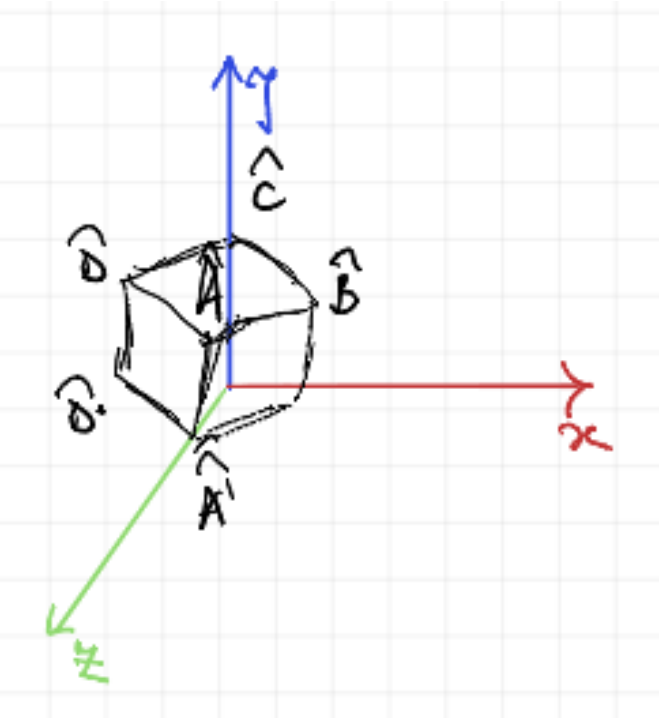
Rotația trecuie făcută prima - ea este facută în jurul axei O_z globală.

Cubul trebuie să fie **în origine** când facem rotația!

Dacă rotim după translație vom roti în spațiul global și poziția cubului.

Rotația

executată înainte de translație



Vom folosi matricea de rotație după axa O_y cu un unghi de 45°.

$$R_y = \begin{bmatrix} 0.7 & 0 & 0.7 & 0 \\ 0 & 1 & 0 & 0 \\ -0.7 & 0 & 0.7 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

adică

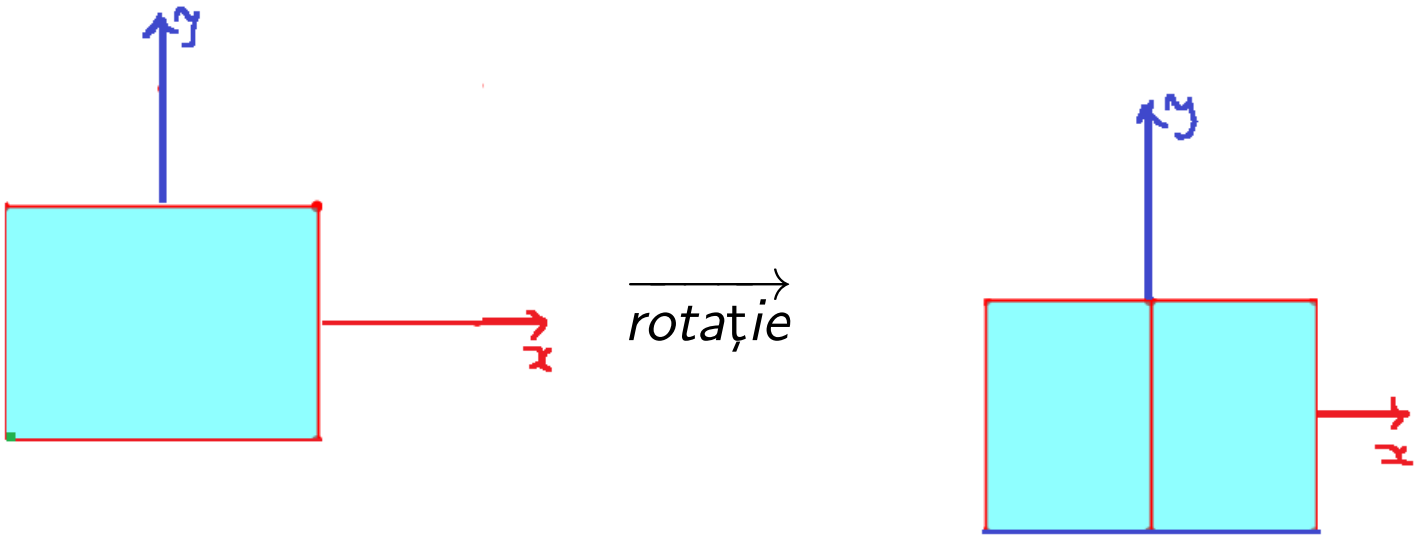
$$\hat{A} = R_y \cdot A = \begin{bmatrix} 0.7 & 0 & 0.7 & 0 \\ 0 & 1 & 0 & 0 \\ -0.7 & 0 & 0.7 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} -0.5 \\ 0.5 \\ 0.5 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.5 \\ 0.7 \\ 1 \end{bmatrix}$$

Similar se face pentru celelalte puncte.

Ce se vede

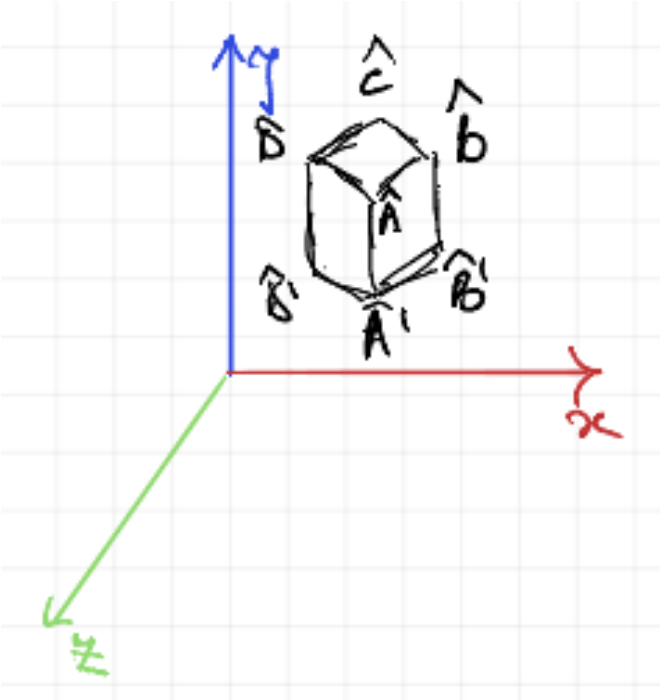
după rotație

Atentie! Observatorul e în mijlocul ecranului și nu avem nici o perspectivă încă!



Translația

făcută după rotație



Vom folosi matricea de translație

$$T = \begin{bmatrix} 1 & 0 & 0 & 0.5 \\ 0 & 1 & 0 & 0.5 \\ 0 & 0 & 1 & -4 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

adică

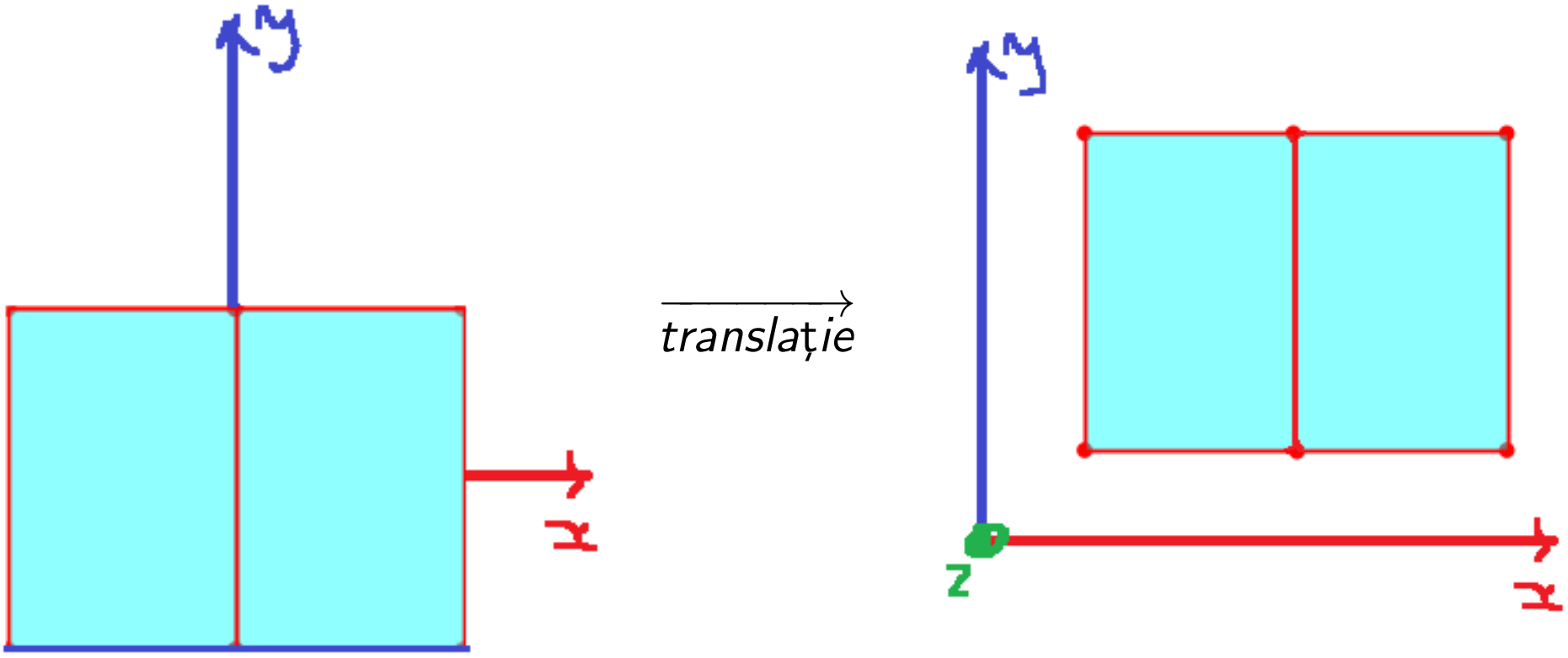
$$\bar{A} = T \cdot \hat{A} = \begin{bmatrix} 1 & 0 & 0 & 0.5 \\ 0 & 1 & 0 & 0.5 \\ 0 & 0 & 1 & -4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0.5 \\ 0.7 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 1 \\ -3.3 \\ 1 \end{bmatrix}$$

Similar se face pentru celelalte puncte.

Ce se vede

după translație și rotație

Atentie! Observatorul e în mijlocul ecranului și nu avem nici o perspectivă încă!



Compunerea transformărilor

Transformările sunt modelate prin înmulțiri matriciale:

$$\hat{A} = R_y \cdot A$$
$$\bar{A} = T \cdot \hat{A}$$

compuse vor fi:

$$\bar{A} = T \cdot \hat{A} = T \cdot R_y \cdot A$$

Matematic înmulțim matricile în **ordine inversă** a transformărilor!

Exemplu de compunere

$$\bar{A} = T \cdot \hat{A} = T \cdot R_y \cdot A = \begin{bmatrix} 1 & 0 & 0 & 0.5 \\ 0 & 1 & 0 & 0.5 \\ 0 & 0 & 1 & -4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0.7 & 0 & 0.7 & 0 \\ 0 & 1 & 0 & 0 \\ -0.7 & 0 & 0.7 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} -0.5 \\ 0.5 \\ 0.5 \\ 1 \end{bmatrix} =$$

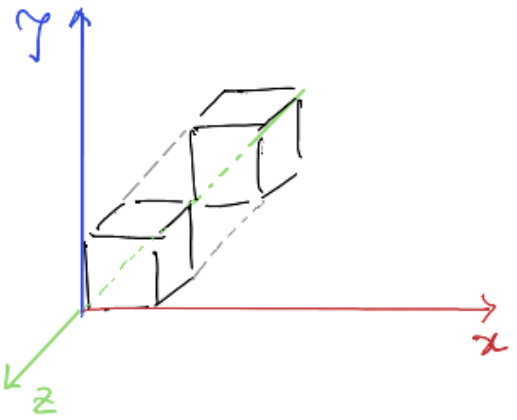
$$= \begin{bmatrix} 0.7 & 0 & 0.7 & 0.5 \\ 0 & 1 & 0 & 0.5 \\ -0.7 & 0 & 0.7 & -4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} -0.5 \\ 0.5 \\ 0.5 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 1 \\ -3.3 \\ 1 \end{bmatrix}$$

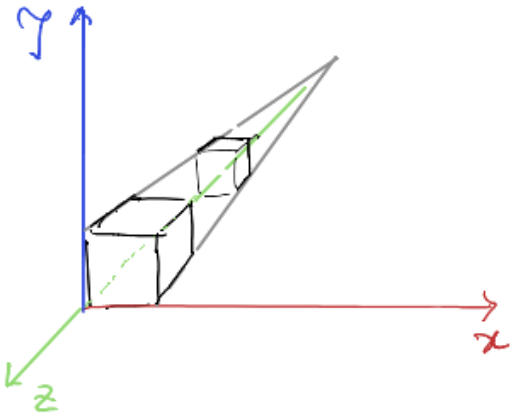
Coordonatele de vizualizare

Se vede clar că pentru o imagine corectă mai trebuie făcute transformări.

Avem două variante:

- perspectivă **ortografică** $\longrightarrow$ imaginile sunt compuse ca în desenul tehnic (folosit când ați învățat geometria). Obiectele din adâncime au aceeași dimensiune ca cele din primele planuri;
- perspectivă **reală** $\longrightarrow$ imaginile sunt compuse ținând seama că obiectele mai îndepărtate sunt percepute ca fiind mai mici.



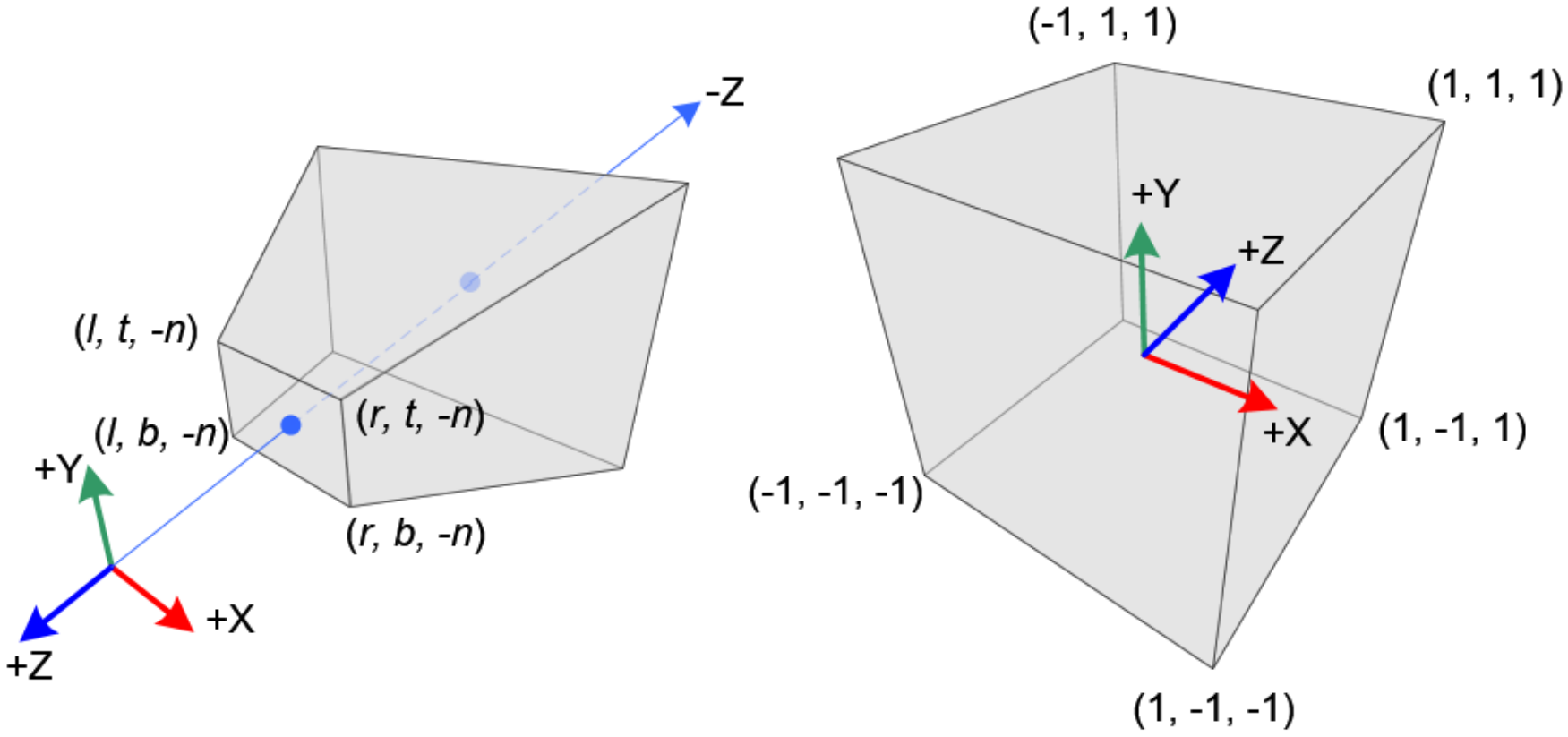


Vom avea o perspectivă reală

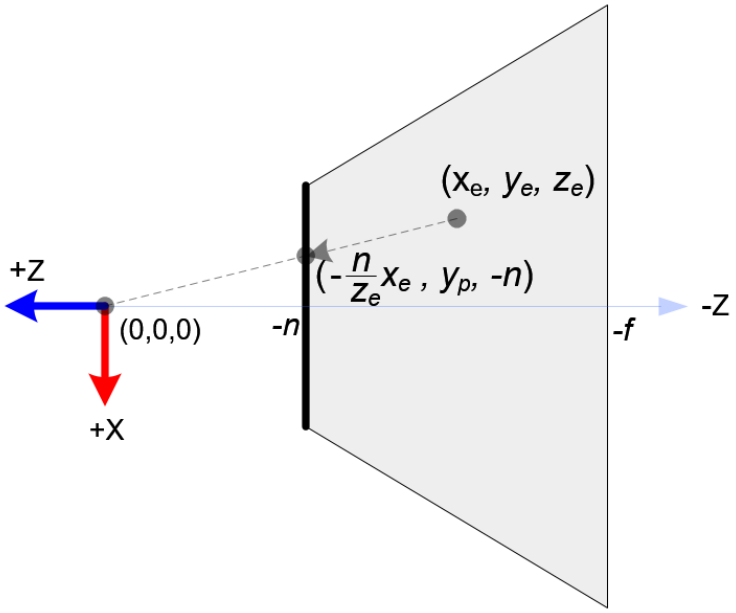
Frustrum

Frustrum-ul

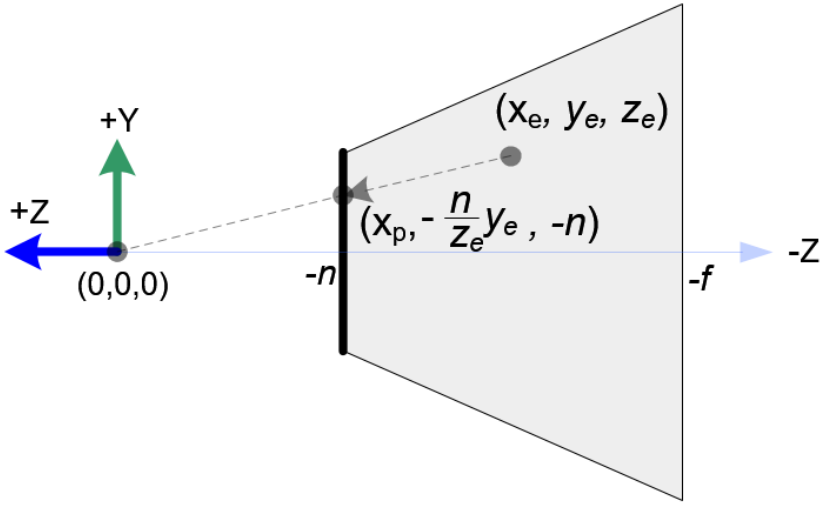
este regiunea spațiului din lumea modelată care va apărea pe ecran.



Calculul coordonatelor



Vedere de sus a Frustum-ului



Vedere laterală a Frustum-ului

Amplasarea Observatorului
(camera)

Vom așeza un observator (o cameră video) din persepectiva căruia se va vedea scena noastră.

Determinăm cei patru vectori R, U, D, P pentru a crea matricea $LookAt$:

$$LookAt = \begin{bmatrix} R_x & R_y & R_z & 0 \\ U_x & U_y & U_z & 0 \\ D_x & D_y & D_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 & -P_x \\ 0 & 1 & 0 & -P_y \\ 0 & 0 & 1 & -P_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Coordonatele Ochiului

- sunt din punctul de observație a privitorului.
- reprezintă coordonate virtuale folosite ca referință.
- le privim ca coordonate pe ecran *absolute*.

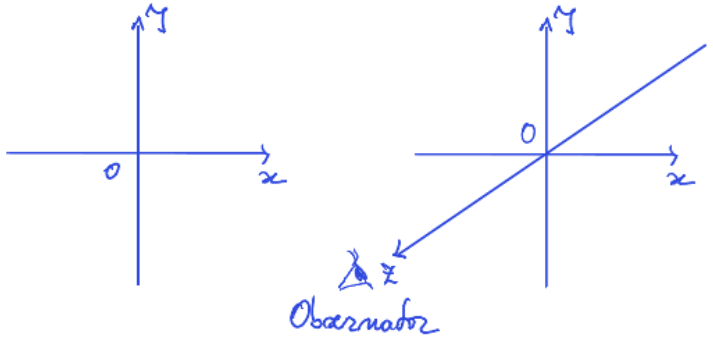


Figure: Centrul sistemului este în punctul $O = (0, 0, 0)$ în mijlocul ecranului.

Tot ce se găsește in partea cu $z - negativ$ a spațiului se vede și tot ce se găsește in partea cu $z - pozitiv$ e în spatele observatorului și nu se vede.

Camera

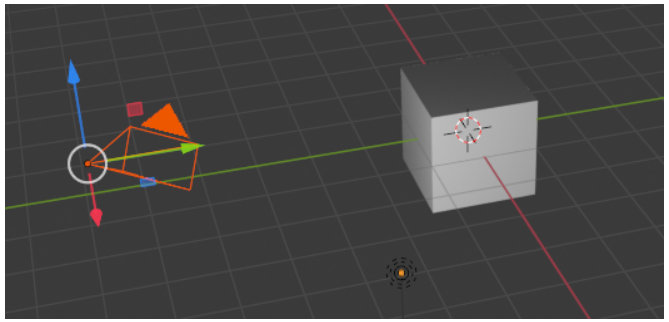


Figure: Camera în Blender. Observați sistemul de coordonate al camerei.

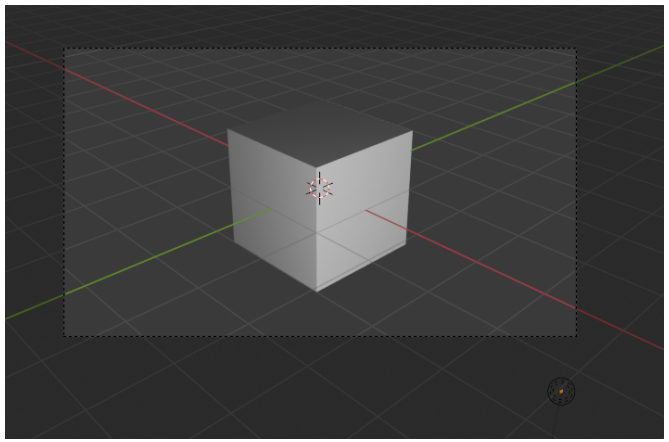
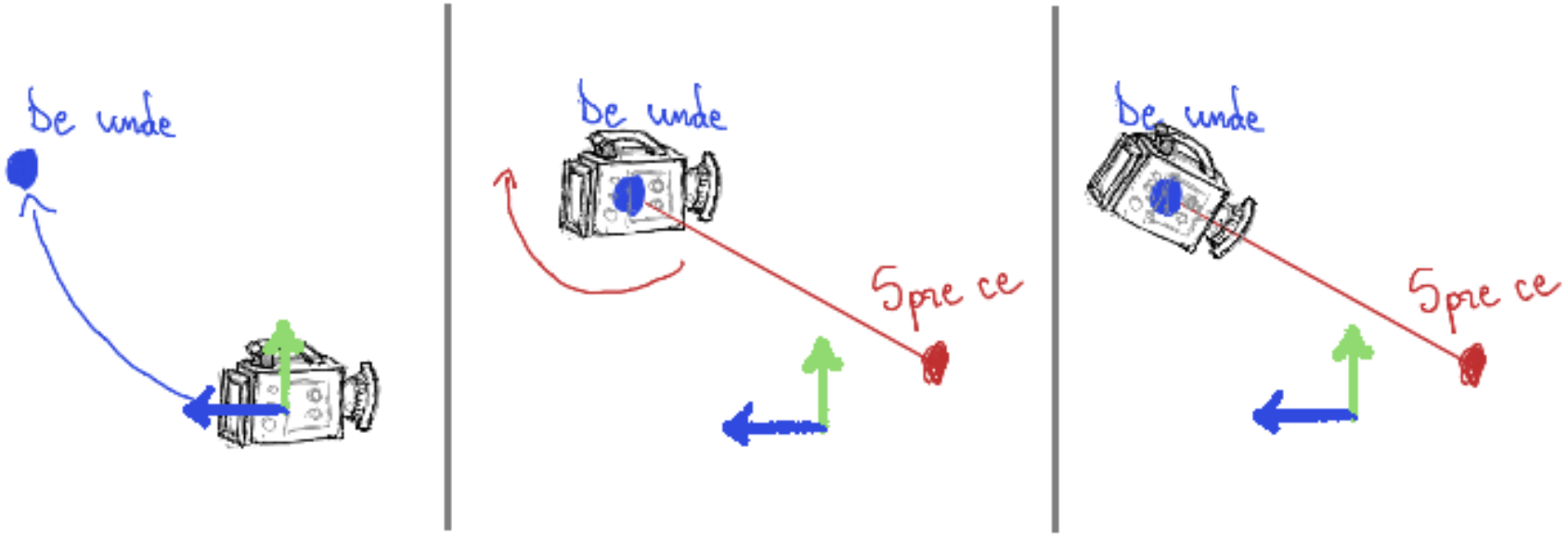


Figure: Cum se vede cubul prin camera respectivă. Zona luminată va fi proiecția pe ecran.

Amplasarea camerei

Metoda "**look-at**"



1. Amplasăm camera la coordonatele de unde vrem să ne uităm.
2. Rotim camera astfel încât să fie aliniată cu direcția spre coordonatele punctului unde vrem să ne uităm.

Amplasarea camerei

Avem 2 sisteme de coordonate:

- unul global
- unul legat de camera

Poziția camerei este un vector în spațiu care indică poziția camerei relativ la sistemul de coordonate global:

$$\overrightarrow{cameraPos} = \overrightarrow{de\ unde}$$

Punctul spre care ne uităm este:

$$\overrightarrow{cameraTarget} = \overrightarrow{spre\ ce}$$

Amplasarea camerei

Direcția camerei

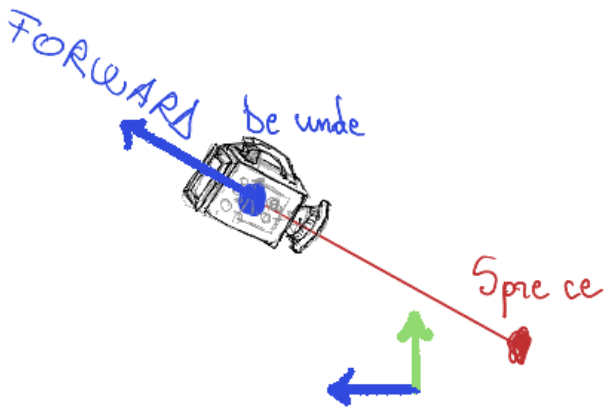


Figure: Calculăm vectorul FORWARD sau cameraDirection din poziția camerei și a punctului țintă.

$$\overrightarrow{cameraDirection} = normalizat(\overrightarrow{cameraPos} - \overrightarrow{cameraTarget})$$

Amplasarea Camerei

Axa spre dreapta relativ la cameră

Un vector spre dreapta care reprezintă axa x pozitivă relativ la cameră.

Versorul axei Oy a sitemului global de coordonate este $\overrightarrow{up} = (0, 1, 0)$.

$$\overrightarrow{cameraRight} = \overrightarrow{up} \times \overrightarrow{cameraDirection}$$

Deoarece rezultatul unui produs vectorial este un vector perpendicular pe ambii vectori, vom obține un versor care indică în direcția pozitivă a axei x.

Dacă am schimba ordinea încrucișată a produsului, am obține un vector care indică axa x negativă.

Amplasarea Camerei

Direcția în sus relativ la cameră

Vectorul care indică axa y pozitivă a camerei este relativ ușor: luăm produsul vectorial al vectorului spre dreapta și a direcției camerei.

$$\overrightarrow{cameraUp} = \overrightarrow{cameraDirection} \times \overrightarrow{cameraRight}$$

Putem acum să construim matricea ce o vom folosi pentru transformarea ce ne va da imaginea ce o putem vedea cu camera proaspăt amplasată.

View matrix

Avem două posibilități de a construi această matrice. Depinde de API-ul folosit.

- Camera-To-World
- World-To-Camera (DirectX, Vulkan, sau Metal)

Bibliografie

www.scratchapixel.com
www.opengl.org
learnopengl.com